

An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

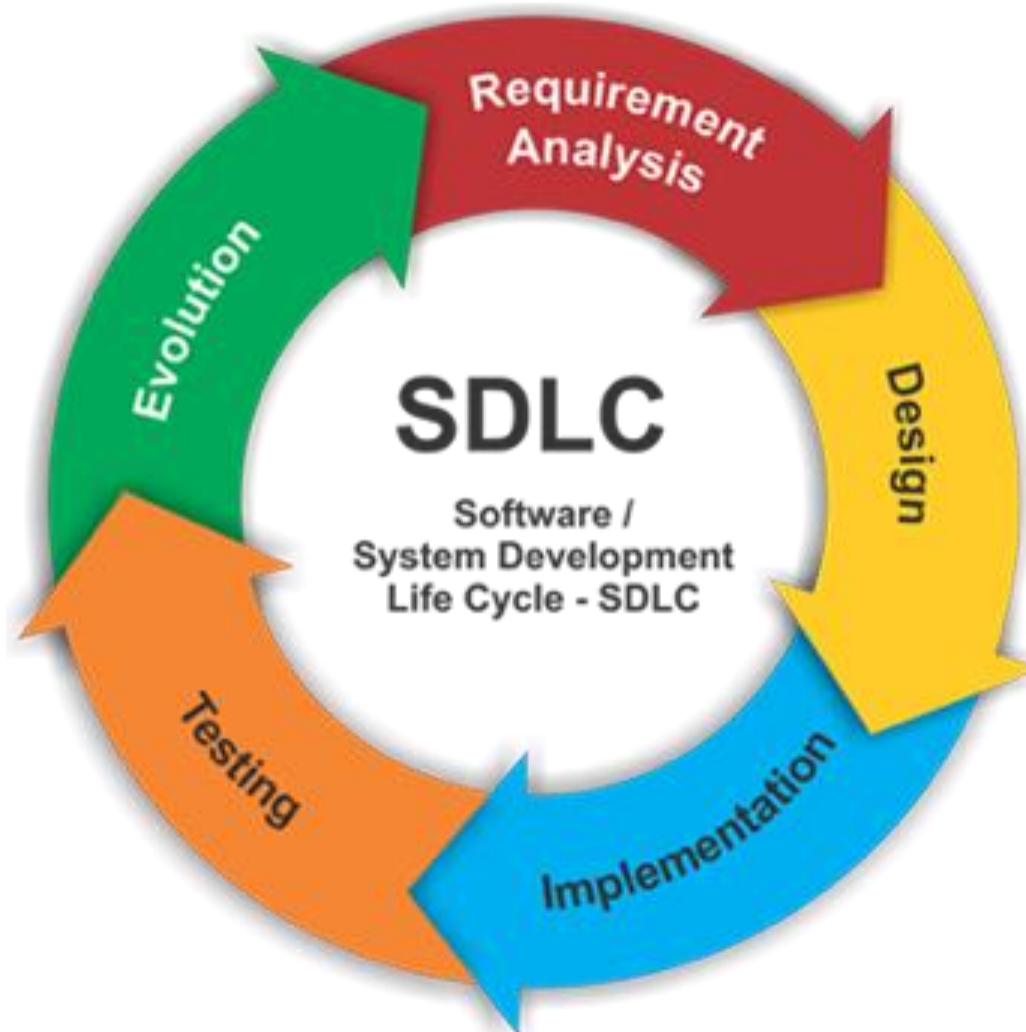
Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Class Activity #2 – Polymorphism



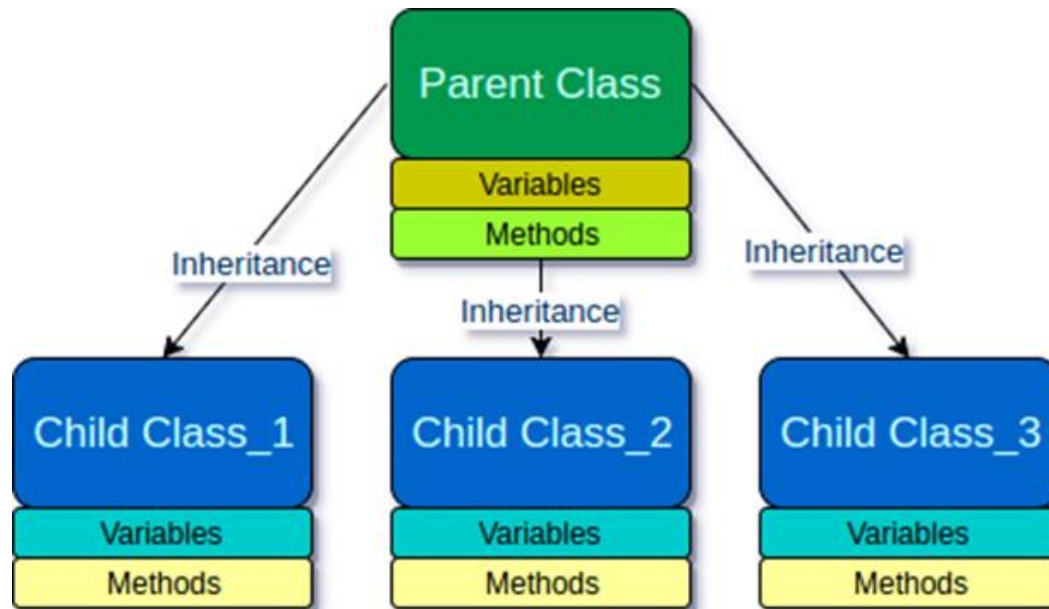
**Welcome to *the* Object Oriented
Programming *course***

Software Development Life Cycle (SDLC) is a structured process that enables the production of high-quality, low-cost software, in the shortest possible production time.



Parent Class vs Child Class

Class child diwarisi seluruh method dari *parent class*. Tapi, ada beberapa kasus di mana **metode tersebut tidak cocok** dengan *child class*. Oleh karena itu, kita harus mengimplementasikan (*re-used*) kembali metode yang pada *child class* yang dinamakan **Method Overriding**.



Polymorphism is a concept where objects of different classes can have different behaviors when calling the same method.

[1] Polymorphism *with* Inheritance

```
# create parent class
class Bird:
    def intro(self):
        print("There are several different types of bird species.")

    def fly(self):
        print("Almost all birds can fly, but there are some that cannot.")

# create child class 1
class Eagle(Bird):
    def fly(self):
        print("Yes, an eagle can fly.")

# create child class 2
class Weka(Bird):
    def fly(self):
        print("No, an Weka can't fly.")

# # object of parent class & child class created
obj_burung = Bird()
obj_elang = Eagle()
obj_weka = Weka()

# user-defined function is called from obj_burung (parent class)
obj_burung.intro()
obj_burung.fly()

# user-defined function is called from obj_elang (child class)
obj_elang.intro()
obj_elang.fly()

# user-defined function is called from obj_weka (child class)
obj_weka.intro()
obj_weka.fly()
```



The diagram illustrates the inheritance hierarchy using red curly braces and numbers. A large brace labeled '1' groups the `Bird` class and its `intro` and `fly` methods. A second brace labeled '2' groups the `Eagle` class and its `fly` method. A third brace labeled '3' groups the `Weka` class and its `fly` method. This indicates that `Eagle` inherits from `Bird`, and `Weka` inherits from `Bird`, with each child class potentially overriding the `fly` method.

- ❖ Polymorphism in object-oriented programming with Python defines a *child class* that has the same method names as the *parent class*. In inheritance, the **child class** inherits methods from the **parent class**. In addition, **it is possible to modify methods** in the child class that have been inherited from the parent class.
- ❖ Polymorphism pada pemrograman berorientasi objek dengan Python mendefinisikan *child class* yang memiliki kesamaan nama metode pada *parent class*. Pada inheritance, *child class* mewarisi metode dari *parent class*.
- ❖ Selain itu, **memungkinkan untuk memodifikasi metode** di dalam child class yang telah diwarisi dari parent class.

[2] Polymorphism *with* Class

```
# create class 1
class Cat:
    # constructor (initialize instance variable)
    def __init__(self, nama, umur):
        self.nama = nama
        self.umur = umur

    def speak(self):
        print("Cat speaks Meow Meow Meow.")

# create class 2
class Dog:
    # constructor (initialize instance variable)
    def __init__(self, nama, umur):
        self.nama = nama
        self.umur = umur

    def speak(self):
        print("Dog speaks Gukk Gukk Gukk.")

# object of Cat created
kucing1 = Cat("Catty", 5)

# object of Dog created
anjing1 = Dog("Doggy", 4)

# memanggil metode tanpa mengabaikan objek
kucing1.speak()
anjing1.speak()

# memanggil metode dengan mengabaikan objek
for hewan in (kucing1, anjing1):
    hewan.speak()
```

- ❖ We can use the concept of polymorphism when creating methods for a class. Python allows different classes to use methods with the same name. We can then call the method by ignoring the object we are using.
- ❖ Kita dapat menggunakan konsep polymorphism ketika membuat metode sebuah Class.
- ❖ Bahasa pemrograman Python memungkinkan sebuah class yang berbeda untuk menggunakan metode dengan nama yang sama.
- ❖ Kemudian kita dapat memanggil metode tersebut dengan mengabaikan objek yang sedang kita gunakan.

[3] Access Variable & Method

```
demo3.py X
1 # create parent class
2 class Bird:
3     # variable
4     color = "red";
5
6     def intro(self):
7         print("There are several different types of bird species.")
8
9     def fly(self):
10        print("Almost all birds can fly, but there are some that cannot.")
11
12 # create child class 1
13 class Eagle(Bird):
14     # variable
15     color = "blue";
16
17     def fly(self):
18         print("Yes, an eagle can fly.")
19
20 # create child class 2
21 class Weka(Bird):
22     # variable
23     color = "green";
24
25     def fly(self):
26         print("No, an Weka can't fly.")
27
28 # # object of parent class & child class created
29 obj_burung = Bird()
30 obj_elang = Eagle()
31 obj_weka = Weka()
32
33 # user-defined function is called from obj_burung (parent class)
34 obj_burung.intro()
35 obj_burung.fly()
36 print("Warna burung: {}".format(obj_burung.color))
37
38 # user-defined function is called from obj_elang (child class)
39 obj_elang.intro()
40 obj_elang.fly()
41 print("Warna burung: {}".format(obj_elang.color))
42
43 # user-defined function is called from obj_weka (child class)
44 obj_weka.intro()
45 obj_weka.fly()
46 print("Warna burung: {}".format(obj_weka.color))
```

- ❖ Terdapat nama **variable** dan **method** yang sama di masing class.
- ❖ Pada inheritance, child class mewarisi metode dari parent class.
- ❖ Itu memungkinkan **untuk memodifikasi variable dan metode** di dalam *child class* yang telah diwarisi dari *parent class*.

Istilah "**property**", "**attribute**", dan "**method**" di OOP sering digunakan untuk menggambarkan ***karakteristik*** atau **perilaku** dari *object* dalam sebuah class.

A **property** is data associated with an object in a class and often represents the attributes attached to that object. An example of a property could be a **variable** that stores object-specific values.

The more general term **attribute** is often used to address both properties and methods in a class. Attribute includes everything that is attached to the object in the class, such as data (*property*) or behavior (*method*).

Methods are functions that exist in a class. Method is used to manipulate object data, perform certain operations on the object, or return values based on the state of the object. Methods have access to properties and other methods in the class.

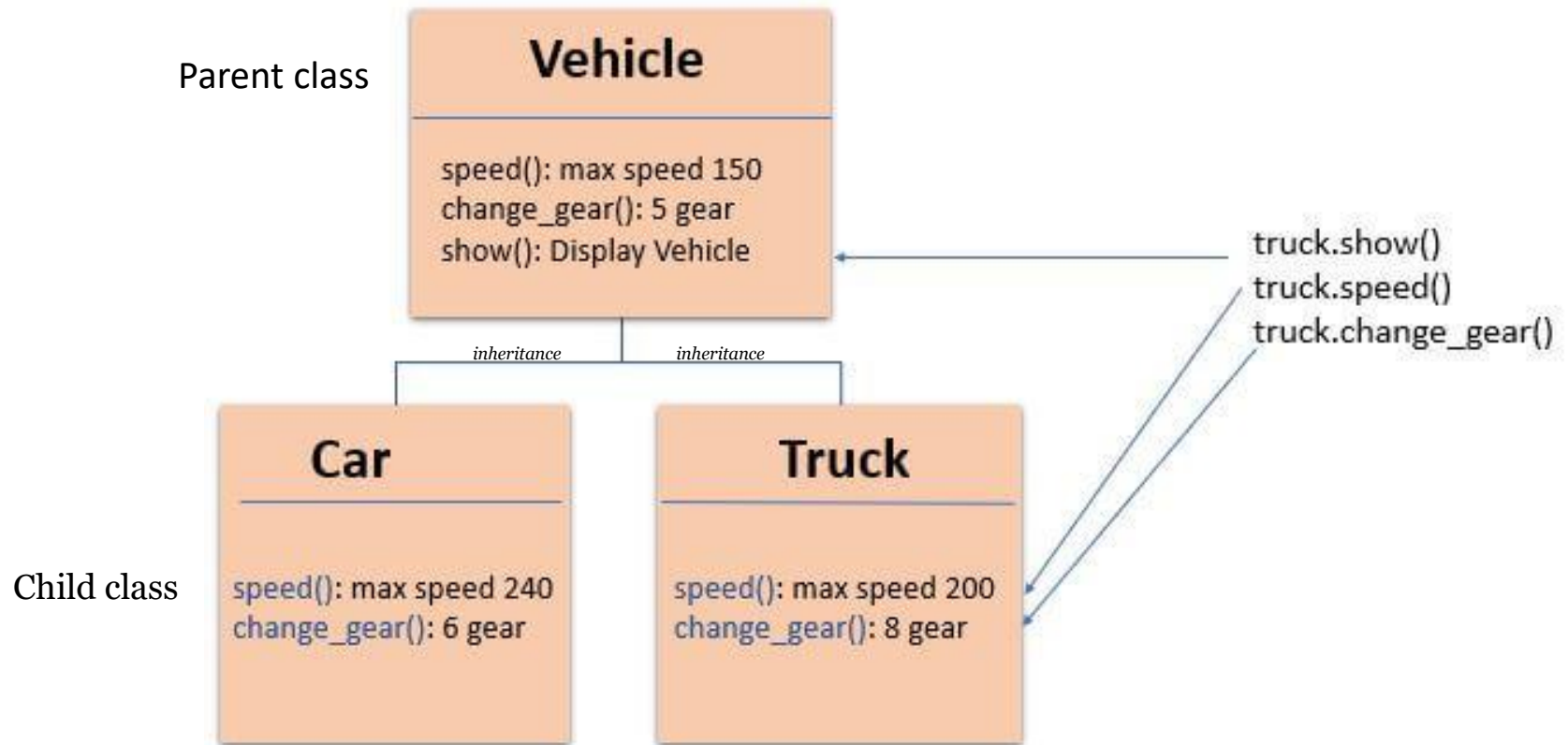


Follow the steps on the next slide!!!

The image features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text "Class Activity" is centered in a large, black, serif font.

Class Activity

Class Diagram #1



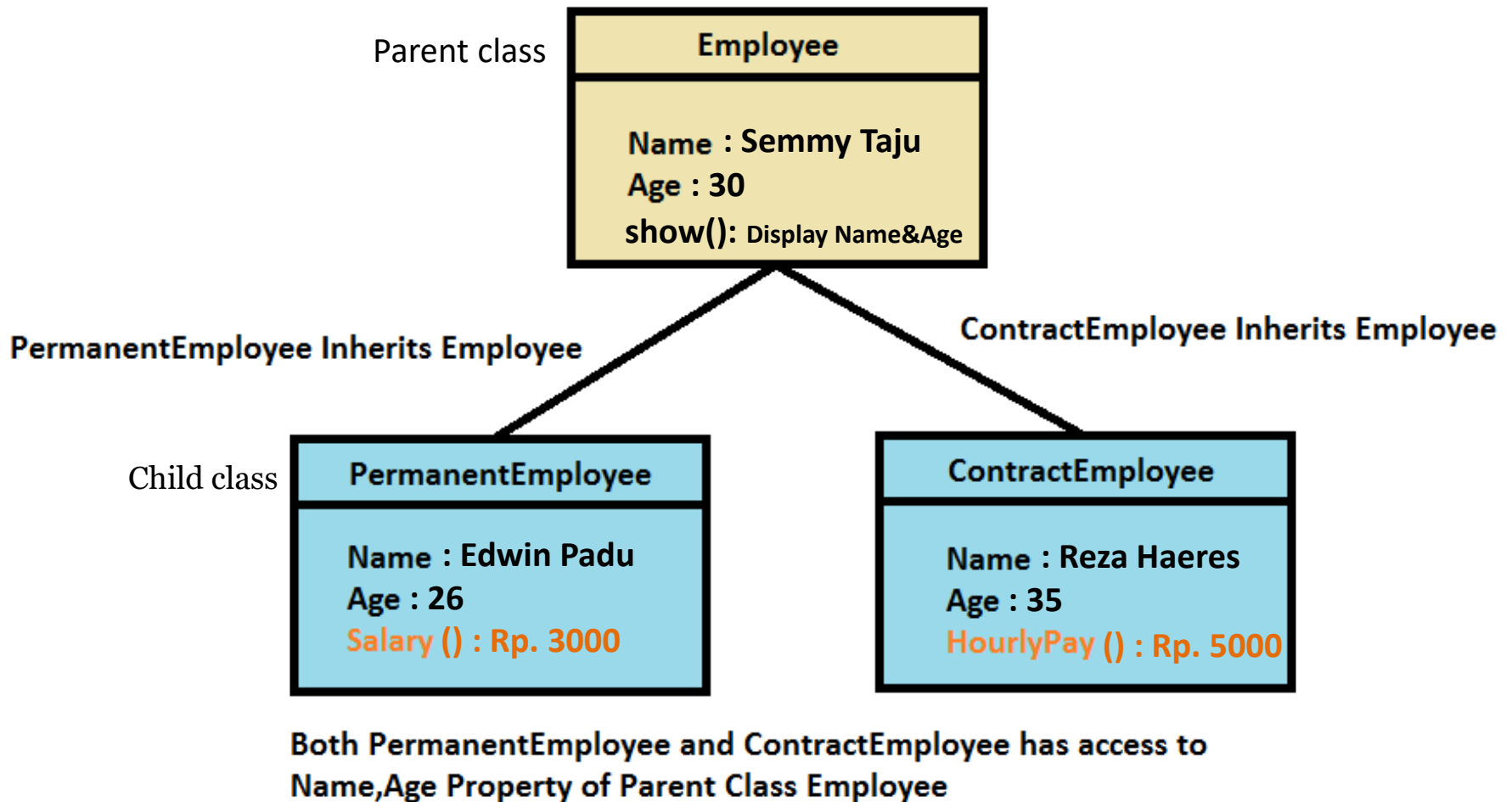
- ❖ Child class Car diwarisi seluruh method dari parent class Vehicle.
- ❖ Child class Truck diwarisi seluruh method dari parent class Vehicle.
- ❖ Keduanya, class Car dan class Truck mempunyai akses ke setiap method dari parent class Vehicle.

Berapa method yang bisa di akses dari class Car dan class Truck???

Instruksi #1

- 1) Setiap *student* mampu untuk mengubah dari bentuk Class Diagram ke dalam bentuk *source code* menggunakan Python code.
- 2) Terdapat *tiga class* yang akan dibuat, **satu *parent class*** dan **dua *child class***.
 - 1) Child class Car diwarisi seluruh method dari parent class Vehicle.
 - 2) Child class Truck diwarisi seluruh method dari parent class Vehicle.
 - 3) Keduanya, class Car dan class Truck mempunyai akses ke class *variable* dan method dari parent class Vehicle.
- 3) Students diminta untuk membuat *object* dari masing-masing class tersebut, baik itu *parent class* dan juga *child class*.
- 4) Setiap method (*user-defined function*) yang bisa di akses (*semua*) akan dipanggil dari masing-masing *object* yang telah dibuat.
- 5) Tampilkan output program.

Class Diagram #2



Berapa method yang bisa di akses dari class Employee???

Instruksi #2

- 1) Setiap *student* mampu untuk mengubah dari bentuk Class Diagram ke dalam bentuk *source code* menggunakan Python code.
- 2) Terdapat tiga class yang akan dibuat, **satu parent class** dan **dua child class**.
 - 1) Child class PermanentEmployee diwarisi seluruh method dari parent class Employee.
 - 2) Child class ContractEmployee diwarisi seluruh method dari parent class Employee.
 - 3) Keduanya, class PermanentEmployee dan class ContractEmployee mempunyai akses ke *variable* dan *method* dari parent class Employee.
- 3) Students diminta untuk membuat *object* dari masing-masing class tersebut, baik itu *parent class* dan juga *child class*.
- 4) Setiap method (*user-defined function*) yang bisa di akses (*semua*) akan dipanggil dari masing-masing *object* yang telah dibuat.
- 5) Tampilkan output program.

Today Report

- 1) Screenshot code program yang sudah dibuat dan jelaskan dengan menggunakan nomor yang jelas.
- 2) Screenshot output program.
- 3) Submit dalam format PDF.
- 4) Batas waktu submission: Jam class

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

